

MODULE 10

Model Selection and Evaluation

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Model Selection and Evaluation

Train-Test Split

```
from sklearn.model_selection import train_test_split

# Simple split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Stratified: preserves class distribution
# random_state: reproducibility

print(f'Train size: {len(X_train)}')
print(f'Test size: {len(X_test)}')
print(f'Train class dist: {pd.Series(y_train).value_counts()}')
print(f'Test class dist: {pd.Series(y_test).value_counts()}')
```

Cross Validation

```
from sklearn.model_selection import cross_val_score, KFold, StratifiedKFold

# 5-fold cross validation
cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f'CV Mean: {cv_scores.mean():.4f} +/- {cv_scores.std():.4f}')

# Stratified K-Fold for classification
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
    X_train, X_val = X[train_idx], X[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]
    model.fit(X_train, y_train)
    print(f'Fold {fold+1}: {model.score(X_val, y_val):.4f}')
```

Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier

# Grid Search (exhaustive)
param_grid = {'n_estimators': [100, 200, 300], 'max_depth': [5, 10, None]}
grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5, scoring='f1')
grid.fit(X_train, y_train)
print('Best params:', grid.best_params_)

# Random Search (faster)
from scipy.stats import randint
param_dist = {'n_estimators': randint(100, 500), 'max_depth': randint(3, 15)}
rand = RandomizedSearchCV(RandomForestClassifier(), param_dist, n_iter=20, cv=5)
rand.fit(X_train, y_train)
```

Overfitting and Underfitting

Underfitting (High Bias):

- Model too simple
- Poor train AND test performance
- Fix: more features, complex model, less regularisation

Overfitting (High Variance):

- Model memorises training data
- Great train, poor test performance
- Fix: more data, regularisation, dropout, cross-validation

Bias-Variance Trade-off:

Total Error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$

Model Comparison

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
```

```
models = {
    'Logistic': LogisticRegression(),
    'RandomForest': RandomForestClassifier(),
    'GradientBoost': GradientBoostingClassifier(),
    'SVM': SVC()
}
```

```
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=5)
    print(f'{name}: {scores.mean():.4f} +/- {scores.std():.4f}')
```